



**INSTITUTO FEDERAL DE CIÊNCIAS E TECNOLOGIA DO PIAUÍ
- CAMPUS PEDRO II
CURSO TÉCNICO EM INFORMÁTICA**

DOCUMENTO DE REQUISITOS

nome da solução

CLINIQ

área temática

Saúde

Equipe:

João Renato Melo Brito
Amanda Ferreira Damasceno
Maria Iasmin Lima Amaral

1. Visão Geral do Sistema

O **CliniQ** é um sistema de organização médica destinado a clínicas, hospitais e pacientes, com o objetivo de centralizar laudos, exames e documentos médicos em formato digital. Ele visa resolver problemas comuns na saúde, como extravio de papéis, dificuldade de acesso rápido ao histórico médico e falta de integração entre laboratórios e pacientes. Em sistemas modernos de saúde, é considerado fundamental permitir o armazenamento eletrônico de documentos e exames para otimizar o atendimento e melhorar a eficiência.

Por meio do **CliniQ**, os usuários podem digitalizar ou importar exames, organizar seus documentos por data e categoria (por exemplo, exames de sangue, imagens, etc.), compartilhar resultados com profissionais de saúde e até mesmo receber lembretes de novos exames ou verificações futuras.

1.1 Principais funcionalidades e casos de uso incluem:

- **Cadastro de Usuários:** Permitir o cadastro de diferentes perfis de usuário – Paciente, Médico e Laboratório – com controles de acesso apropriados para cada tipo.
- **Upload/Adição de Exames:** Pacientes ou laboratórios podem adicionar (fazer upload) de exames e documentos médicos ao sistema, que ficarão armazenados digitalmente associados ao paciente correto.
- **Visualização de Exames:** Usuários autorizados (médicos, pacientes e laboratórios) podem visualizar os exames armazenados, acessando imagens ou laudos em formato digital diretamente pelo sistema.
- **Validação de Exames:** Médicos ou laboratórios podem validar de forma simples um exame enviado, confirmando a veracidade ou adequação do documento anexado (aprovação do laudo). Obs.: Nesta versão do projeto a validação é interna e manual (pelo profissional), mas futuramente poderia ser integrada a sistemas externos de autenticação de laudos.
- **Compartilhamento Controlado:** O paciente pode compartilhar seus exames com profissionais de saúde de forma controlada, escolhendo quais documentos compartilhar e com qual médico ou laboratório, assegurando que apenas os destinatários autorizados tenham acesso.
- **Linha do Tempo de Eventos Médicos:** O sistema apresenta uma linha do tempo visual dos eventos médicos do paciente, mostrando cronologicamente acontecimentos como uploads de exames, validações e compartilhamentos, facilitando o acompanhamento do histórico de saúde.

Em suma, o **CliniQ** proporciona aos pacientes um prontuário eletrônico pessoal e unificado – reunindo seus exames e documentos médicos importantes – e aos médicos e laboratórios uma forma ágil de acessar e validar essas informações. Trata-se de um projeto educacional de porte simples (nível ensino médio), focado em demonstrar conceitos de organização de informação médica digital, sem intuito comercial ou de monetização.

2. Requisitos do Sistema

Nesta seção são detalhados os requisitos do sistema CliniQ, divididos em funcionais (funcionalidades e comportamentos esperados) e não funcionais (qualidades, restrições e tecnologias do sistema).

2.1 Requisitos Funcionais (RF)

A seguir estão listados os requisitos funcionais essenciais do CliniQ, numerados para referência:

RF001 – Cadastrar Usuário: O sistema deve permitir o cadastro de novos usuários dos três tipos: Paciente, Médico ou Laboratório. Cada usuário terá um perfil com informações básicas (nome, e-mail, etc.) e uma definição de tipo/perfil que determina suas permissões de acesso. Deve haver validação de dados ao cadastrar (por exemplo, evitar e-mails duplicados) e uma forma de autenticação simples (login) para controlar o acesso conforme o tipo de usuário.

RF002 – Adicionar Exame: O sistema deve possibilitar que um Paciente ou Laboratório faça upload de um exame ou documento médico. Isso inclui fornecer uma interface para envio do arquivo (por exemplo, PDF ou imagem do laudo) e preenchimento de metadados como título, data do exame, tipo de exame (categoria) e possivelmente observações. O exame adicionado deve ficar associado a um paciente específico no sistema (no caso de upload feito por um laboratório, o laboratório deverá indicar a qual paciente o exame pertence). Após o upload, o arquivo deve ser armazenado de forma segura (vide RF004) e um registro do exame é criado no prontuário do paciente.

RF003 – Validar Exame: O sistema deve permitir a validação simples de um exame digital enviado, realizada por um Médico ou um usuário de Laboratório autorizado. Na prática, isso significa que um profissional de saúde pode marcar um exame como “validado” ou “autenticado” dentro do sistema, atestando que o documento foi verificado e é confiável. Essa validação estará associada ao usuário que a realizou (por exemplo, identificação do médico que validou) e à data em que ocorreu. Nota: Diferentemente de uma autenticação totalmente automatizada, aqui a validação é manual e interna, conforme o escopo simplificado do projeto. (Futuramente, poderia-se integrar um sistema externo de autenticação para verificar a procedência dos exames automaticamente, mas isso extrapola os objetivos atuais.)

RF004 – Armazenar Arquivos: O sistema deve armazenar os arquivos dos exames de forma segura e confiável. Cada upload de exame deverá ser salvo no servidor backend (por exemplo, em um diretório protegido ou banco de dados) ou em um serviço de nuvem integrado, de modo que possa ser recuperado para visualização/download. Este requisito funcional garante que os documentos enviados permanecerão disponíveis e íntegros para visualização futura, mesmo após o logout do usuário ou uso em diferentes dispositivos.

RF005 – Visualizar Exame: O sistema deve prover meios para visualizar os exames armazenados. Usuários autorizados (o paciente dono do exame, médicos ou laboratórios com acesso ao exame) poderão abrir e ler o conteúdo do documento dentro do sistema. Por exemplo, ao selecionar um exame, o sistema pode exibir a

imagem ou PDF do laudo em uma tela ou permitir seu download (ver RF006). Devem ser respeitadas as permissões: pacientes veem apenas seus próprios exames; médicos veem apenas exames de pacientes que os compartilharam com eles; laboratórios veem os exames que enviaram ou que lhes foram compartilhados. A interface de visualização deve ser simples, podendo incluir zoom em imagens, paginação de PDFs ou outras facilidades básicas de leitura.

RF006 – Download de Exame: O sistema deve permitir que o usuário baixe uma cópia do arquivo do exame para seu dispositivo, para visualização offline ou arquivamento pessoal. Essa funcionalidade deve estar disponível somente para usuários que tenham permissão de acesso ao exame (por exemplo, o próprio paciente ou um médico com quem foi compartilhado). Ao acionar o download, o sistema fornece o arquivo original enviado (ou uma versão PDF equivalente) para o usuário. Esse requisito garante que os pacientes possam ter posse local de seus documentos e que médicos possam guardar resultados, se necessário, para consulta fora do sistema.

RF007 – Organizar e Buscar Exames: O sistema deve oferecer recursos para organização dos exames/documentos médicos armazenados. Isso inclui, por exemplo, permitir que o paciente (ou seu responsável) classifique ou filtre seus exames por categoria, data ou tipo de exame, ou associe etiquetas (tags) descritivas a cada documento. Também é desejável uma funcionalidade de busca, onde o usuário possa pesquisar um exame pelo nome, tipo (ex: "receita", "exame de imagem") ou até pelo profissional/laboratório relacionado. Esses recursos de organização facilitam encontrar rapidamente um documento específico dentro de um histórico extenso. Para fins de simplicidade, podem ser implementados como opções de ordenação (por data, tipo) e filtros na listagem de exames do paciente.

RF008 – Compartilhar Exames: O sistema deve permitir o compartilhamento controlado de exames entre usuários. Especificamente, um Paciente pode selecionar um exame do seu prontuário e compartilhá-lo com um Médico (ou, eventualmente, com um Laboratório) de sua escolha. O compartilhamento dá ao profissional selecionado permissão de visualizar (e possivelmente baixar) aquele documento. Deve haver mecanismos para garantir o controle: por exemplo, o paciente deve conseguir revogar um compartilhamento a qualquer momento, removendo o acesso do médico ao seu exame. Poderiam também ser implementadas restrições de tempo (ex.: acesso válido por X dias) ou notificações de compartilhamento, mas no escopo básico basta que o paciente controle quem pode ver cada exame. O sistema deve registrar quando um exame foi compartilhado e com quem, para fins de auditoria e para refletir esses eventos na linha do tempo (RF009). Exemplo: um paciente compartilha seu exame de sangue com o Dr. João, médico clínico – a partir de então o Dr. João consegue visualizar (e baixar) esse exame quando acessar o sistema.

RF009 – Linha do Tempo de Eventos Médicos: O sistema deve gerar e exibir uma linha do tempo interativa com os principais eventos médicos relacionados ao paciente. Nessa timeline cronológica constarão, por exemplo: data em que um exame foi adicionado (upload), indicando quem adicionou (paciente ou qual laboratório); data em que um exame foi validado, indicando o profissional que validou; e eventos de compartilhamento (por exemplo "Exame X compartilhado com Dr. João em 10/10/2025"). Dessa forma, o paciente (ou médico, ao visualizar o histórico do paciente) tem uma visão organizada e temporal de todo o histórico relevante. A linha do tempo contribui para um prontuário eletrônico mais intuitivo, destacando a evolução do cuidado com o paciente (ela pode incluir também outros eventos futuros, como consultas realizadas, cirurgias, receitas emitidas, etc., mas para este projeto focaremos nos eventos relacionados a exames). A interface deve apresentar os eventos ordenados por data (recentes primeiro ou em ordem crescente) com

identificação clara do tipo de evento e data/hora.

2.2 Requisitos Não Funcionais (RNF)

A seguir são descritos os requisitos não funcionais do CliniQ, ou seja, características de qualidade, restrições de projeto e critérios de aceite globais do sistema:

RNF001 – Segurança: O sistema deverá proteger os dados sensíveis dos usuários (informações pessoais e documentos médicos). Isso inclui autenticação básica para acesso (login por senha) e controles de autorização para que um usuário não acesse dados de outro indevidamente. Toda comunicação deve ser preferencialmente feita via protocolo seguro (HTTPS) para garantir confidencialidade em trânsito. Os arquivos de exames e as senhas dos usuários devem ser armazenados de forma segura

RNF002 – Desempenho: O CliniQ deve ser capaz de carregar as informações de forma rápida, proporcionando uma boa experiência ao usuário. Como lida com documentos potencialmente pesados (imagens de exames, PDFs).

RNF003 – Usabilidade: A interface do usuário deve ser simples, intuitiva e fácil de usar. Considerando o público-alvo (inclusive pacientes possivelmente leigos em tecnologia), o design das telas deve ser claro, com fluxo de navegação fácil. Funções principais (como adicionar exame, compartilhar, visualizar) devem estar destacadas e ser acessíveis em poucos cliques. Além disso, toda a interface estará em língua portuguesa e deverá seguir boas práticas de UX para aplicações web.

RNF004 – Compatibilidade: O frontend web do sistema deve funcionar corretamente nos principais navegadores web modernos e em diferentes dispositivos, tanto em computadores (desktop) quanto em dispositivos móveis (tablets, celulares). Isso implica uso de HTML5/CSS3 responsivo ou layouts adaptativos para diversas resoluções de tela. Não serão utilizadas tecnologias exclusivas de um navegador específico, garantindo compatibilidade cross-browser.

RNF005 – Escalabilidade: A solução deve ter potencial de escalabilidade horizontal e vertical para suportar um aumento no número de usuários e de dados. Embora inicialmente implantado em pequena escala, o design deve considerar a possibilidade de crescimento (por exemplo, separar camadas de aplicação e banco de dados, usar APIs REST que poderiam ser consumidas por futuros apps móveis, etc.). Em nível básico, isso significa usar um banco de dados capaz de lidar com muitos registros e permitir futuras otimizações (PostgreSQL atende bem esse critério) e escrever código claro que possa ser otimizado.

RNF006 – Confiabilidade: O sistema deve ser confiável e evitar perda de dados. Toda transação importante (cadastro, upload de exame, compartilhamento) deve ser devidamente confirmada e persistida. É importante que, uma vez enviado um exame, ele permaneça armazenado de forma íntegra; se ocorrer alguma falha durante upload, o usuário deve ser notificado para tentar novamente, garantindo que só sejam salvos documentos completos. Também deve haver tolerância a falhas simples – por exemplo, se o serviço de e-mail (se houver notificações) falhar, isso não deve impedir o cadastro do usuário.

RNF007 – Manutenibilidade: O sistema deve ser escrito e arquitetado de forma a facilitar atualizações e correções futuras. Isso inclui um código fonte modular seguindo a arquitetura em camadas (ver seção de Arquitetura), o uso de convenções de projeto (ex: padronização de nomes, organização de pacotes) e documentação interna do código. A separação clara de responsabilidades torna mais fácil localizar e corrigir bugs ou adaptar funcionalidades. Além disso, optar por tecnologias amplamente utilizadas (Spring Boot, PostgreSQL, etc.) contribui para a manutenibilidade, pois a equipe de desenvolvimento terá acesso a recursos comunitários e documentação abundante.

3. Tecnologias Utilizadas

Para implementar o CliniQ conforme os requisitos acima, optou-se por uma stack tecnológica simples, robusta e adequada a aplicações web Java de médio porte:

Backend – Spring Boot (Java): O servidor será construído usando o framework Spring Boot, que facilita a criação e o deploy de aplicações web em Java, adotando convenções que reduzem a necessidade de configurações manuais .

O Spring Boot oferece um container web embutido (Tomcat) e suporte nativo ao padrão MVC, organizando a aplicação em controladores, serviços e repositórios de forma limpa. Além disso, serão utilizados módulos Spring como Spring Web (para criar controladores e APIs REST seguindo o padrão MVC) e Spring Data JPA (para facilitar o acesso ao banco de dados relacional de forma orientada a objetos). Essa escolha acelera o desenvolvimento, pois muitas funcionalidades comuns (como segurança básica, conexões com BD, gestão de dependências) já vêm prontas ou configuradas por padrão no Spring Boot.

Banco de Dados – PostgreSQL: Os dados do sistema (usuários, exames, etc.) serão armazenados em um banco de dados relacional PostgreSQL. O PostgreSQL é um SGBD open-source renomado por sua robustez, segurança e conformidade com ACID, sendo adequado para armazenar informações críticas de saúde. A integração com Java/Spring é feita via JDBC/JPA utilizando o driver específico do PostgreSQL.

Em termos de modelagem, serão usadas tabelas para cada entidade principal (ver seção de Entidades) com relacionamentos implementados conforme o diagrama ER. Poderíamos também utilizar recursos do PostgreSQL como armazenamento de documentos em formato binário (BLOBs) ou referências a arquivos externos, dependendo da estratégia de armazenamento escolhida (ver RF004).

Frontend – HTML5/CSS3/JavaScript (puro): A interface do usuário será web, desenvolvida em HTML, CSS e JavaScript simples, sem uso de frameworks front-end modernos. Serão criadas páginas estáticas ou com geração dinâmica simples para as telas de login, cadastro, upload de exame, listagem de exames, etc., estilizadas com CSS para boa apresentação e usando JavaScript para interatividade básica (por exemplo, validação de formulários no cliente, chamadas AJAX para o backend ou atualização dinâmica de partes da página). Essa escolha também garante compatibilidade ampla (RNF005), já que HTML/CSS/JS padrão é suportado pela maioria dos navegadores sem necessidade de instalações adicionais. O frontend comunicará com o backend Spring Boot possivelmente via requisições HTTP REST (por exemplo, usando fetch API do JS para enviar formulários e obter dados em JSON), ou através de formulários tradicionais HTTP post/get dependendo da

simplicidade desejada.

Controle de Versão e Build: Opcionalmente, o projeto utilizará Maven ou Gradle (ferramentas de build Java) para gerenciar dependências e empacotamento da aplicação Spring Boot. O código será mantido em um repositório Git para controle de versão durante o desenvolvimento. Essas tecnologias auxiliam na manutenibilidade (RNF008) ao organizar o projeto e permitir reproduções consistentes do ambiente de execução.

Ambiente de Execução: A aplicação Spring Boot será empacotada como um arquivo JAR executável, rodando em um servidor local ou na nuvem (por exemplo, Heroku, AWS ou outro serviço compatível com aplicações Java). O banco de dados PostgreSQL pode ser hospedado localmente ou em um serviço de banco de dados gerenciado. Não serão necessários servidores de aplicação externos, pois o Spring Boot já inclui um contêiner web.

Em resumo, a combinação Spring Boot + PostgreSQL + HTML/JS puro foi escolhida por ser didática e ao mesmo tempo semelhante ao stack usado em aplicações reais, oferecendo uma curva de aprendizado amigável para os desenvolvedores iniciantes e atendendo aos requisitos de forma confiável.

4. Arquitetura em Camadas (MVC)

O CliniQ será desenvolvido seguindo uma arquitetura em camadas, organizada segundo o padrão MVC (Model-View-Controller) tradicional. Essa abordagem separa a aplicação em componentes com responsabilidades distintas, facilitando a manutenção e evolução do sistema.

As principais camadas/partes da arquitetura são:

Camada de Apresentação (View e Controller): Corresponde à interface com o usuário e ao controle das interações. No contexto do CliniQ, inclui as páginas HTML/CSS/JS no navegador (que compõem a View) e os controladores (Controllers) implementados no Spring Boot que recebem requisições HTTP. Os controladores mapeiam URLs para ações do sistema – por exemplo, um `PacienteController` pode definir endpoints para cadastro de paciente, listagem de exames, etc. Essa camada é responsável por receber entradas do usuário (dados de formulários, cliques em botões) e retornar as respostas apropriadas (páginas HTML renderizadas ou dados JSON, dependendo da implementação). Ao seguir MVC, os controladores devem ser simples, delegando a lógica de negócios para a próxima camada.

Camada de Negócio (Model/Service): Aqui residem as regras de negócio e a lógica central da aplicação. Nessa camada teremos os Services do Spring (classes anotadas com `@Service` ou `@Component`), que implementam funcionalidades como validação de exames, verificação de permissões de acesso, processamento de uploads, etc. Por exemplo, um `ExameService` pode conter métodos para salvar um novo exame, realizar a lógica de associá-lo a um paciente e notificar um médico, enquanto um `CompartilhamentoService` gerencia a lógica de compartilhar ou revogar acesso a exames. Essa camada atua como intermediária entre o Controller e os dados, garantindo que todas as regras (p. ex., "somente um médico pode validar um exame") sejam aplicadas corretamente. A Modelagem de Domínio (objetos de negócio) também faz parte desta camada – as classes Modelo (Model) representam as entidades do sistema (Usuário, Exame, etc.) e geralmente estão mapeadas para o banco de dados via JPA. Em suma, a camada de negócio encapsula o que o sistema

faz em termos de processamento, mantendo o Controller focado em quando chamar e a camada de dados focada em como persistir. Essa separação torna o código mais coeso e fácil de manter.

Camada de Dados (Data Access): Responsável pela persistência e recuperação de dados no banco PostgreSQL. É implementada principalmente através dos Repositórios (Repository ou DAO) do Spring Data JPA, que fornecem métodos para operações CRUD nas entidades. Por exemplo, haverá um `UsuarioRepository`, `ExameRepository`, etc., que estendem interfaces do Spring Data JPA (`JpaRepository`) proporcionando métodos como `save`, `findById`, `findAll`, sem necessidade de SQL explícito na maioria dos casos. Essa camada também inclui as configurações de conexão com o banco (`datasource`, credenciais) e eventuais mapeamentos customizados. Ela deve ser acessada somente pela camada de negócio (ou seja, os `Services` chamam os métodos dos Repositórios), garantindo o padrão de arquitetura em camadas onde a de negócio isola a de apresentação dos detalhes de persistência.

5. Entidades do Sistema e seus relacionamentos

Nesta seção, definimos as principais entidades de domínio do CliniQ e como elas se relacionam entre si. Cada entidade corresponde a um conceito ou objeto importante no sistema, e será implementada provavelmente como uma classe Java (anotada como `@Entity` no caso das entidades persistentes) e como uma tabela no banco de dados relacional. A seguir, descrevemos cada entidade:

- **Usuário:** Representa uma pessoa ou organização que utiliza o sistema. Os usuários podem ser de três tipos: Paciente (indivíduo que possui um prontuário de exames), Médico (profissional de saúde que acessa exames de pacientes) ou Laboratório (entidade responsável por realizar exames e enviá-los ao sistema). Embora existam três perfis, optou-se por uma única entidade *Usuário* com um atributo indicativo do tipo/perfil (por exemplo, um campo `tipo` ou três flags booleanas), em vez de tabelas separadas, para simplificar a gestão. A entidade Usuário contém atributos básicos como `id` (identificador único), `nome`, `e-mail` (usado para login) e `senha` (armazenada de forma criptografada/hash) – além do tipo do usuário. Cada usuário do tipo Paciente poderá ter vários exames associados no sistema (relação de um-para-muitos, ver Exame abaixo). Usuários do tipo Médico e Laboratório, por sua vez, não “possuem” exames próprios no sistema, mas podem estar associados a exames seja porque os validaram ou enviaram, ou porque receberam compartilhamento de pacientes. Um Laboratório pode também ser modelado com atributos adicionais (como `nome da instituição`, `endereço`), mas estes detalhes podem ser abstraídos neste projeto simples. No contexto de autenticação, todos os usuários compartilham o mesmo processo de login, diferenciando-se apenas nas autorizações conforme o tipo.
- **Exame (Documento Médico):** Esta entidade representa um exame médico ou documento armazenado no sistema. Ela é central no CliniQ, pois corresponde aos laudos/exames em formato digital que antes existiam apenas em papel. Um objeto Exame contém campos como `id` (identificador), `tipo de exame` (por exemplo: “Hemograma”, “Raio-X”, “Receita Médica”), `data do exame` (data em que foi realizado ou enviado), e uma referência ao arquivo digital em si (por exemplo, um caminho de arquivo ou URL onde o PDF/imagem está armazenado). Além disso, tem atributos de relacionamento: cada Exame está

associado a exatamente um Paciente (o paciente ao qual o exame pertence) – geralmente o paciente que realizou o exame – e portanto armazena um `patientId` que referencia o Usuário correspondente. O Exame também pode guardar quem foi o usuário responsável pelo upload (atributo `uploaderId` – podendo ser o próprio paciente ou um laboratório que adicionou o exame em nome do paciente) e quem foi o validador do exame (atributo `validadorId`, se um médico ou lab já tiver marcado como validado). Cada exame pode opcionalmente ter um status de validado (por exemplo, um campo booleano indicando se foi revisado/validado por um profissional). Resumidamente, a entidade Exame conecta pacientes, profissionais e arquivos de laudo. Em termos de relacionamentos: um Paciente pode ter N exames em seu histórico (1:N), cada Exame pertence a 1 paciente; um Laboratório pode enviar N exames (cada Exame tem no máximo 1 lab uploader); um Médico pode validar N exames (cada Exame tem no máximo 1 validador). Esses relacionamentos de `uploader` e `validador` são implementados via campos estrangeiros que referenciam a tabela de Usuários, mas não representam posse, apenas associação. Para modelar quem tem permissão de acessar cada exame, utilizamos a entidade de Compartilhamento (abaixo). Vale destacar que *Exame* aqui é um termo genérico – podendo englobar também outros documentos médicos como laudos de imagem, receitas, atestados, etc., conforme a necessidade, já que todos são documentos vinculados ao paciente. No diagrama e implementação, porém, todos podem ser tratados sob a mesma entidade Exame, distinguindo pelo campo de tipo.

- **Compartilhamento (Permissão de Acesso):** Para permitir o *compartilhamento controlado* de exames entre usuários (RF008), definimos uma entidade que representa a relação de acesso entre um usuário e um exame que não lhe pertence originalmente. Um *Compartilhamento* registra que um determinado Usuário (por exemplo, um Médico) tem acesso a um determinado Exame de um paciente, possivelmente concedido pelo paciente. Essa entidade pode ser vista como uma tabela de associação entre Exame e Usuário, de natureza *muitos-para-muitos*: um exame pode ser compartilhado com vários usuários (ex.: vários médicos diferentes) e um usuário (médico) pode ter acesso a vários exames de distintos pacientes. Os principais atributos de Compartilhamento são: um `examId` (referência ao Exame compartilhado), um `usuariId` (referência ao usuário com quem foi compartilhado) e possivelmente a data do compartilhamento. O usuário referência normalmente será um médico ou profissional que não seja o dono do exame – não faria sentido registrar compartilhamento do paciente consigo mesmo. Com essa estrutura, consultar quem pode ver um exame ou quais exames um médico pode ver torna-se uma questão de buscar os registros em *Compartilhamento*. Essa tabela também suporta funcionalidades como revogação de acesso (bastando remover o registro correspondente) ou implementação de prazo de validade (adicionando um campo de expiração, por exemplo, embora não exigido neste escopo). Em resumo, a entidade Compartilhamento materializa no banco de dados as permissões concedidas de visualização de exames.
- **Evento (Log de Eventos) (opcional):** Para implementar a Linha do Tempo (RF009), pode-se modelar uma entidade de Evento que registra acontecimentos no sistema, como `UPLOAD_EXAME`, `VALIDACAO_EXAME` ou `COMPARTILHAMENTO_CONCEDIDO`. Cada Evento poderia ter um ID, tipo, timestamp, referência ao exame e usuário envolvido, e descrição. No entanto, para simplicidade, muitos desses dados já existem implícitos: a criação de um Exame já contém a data do upload; a validação pode ser inferida do campo de validado e validador no Exame; e os compartilhamentos são registrados com

data na entidade Compartilhamento. Assim, talvez não seja necessária uma tabela separada de eventos – a timeline pode ser construída consultando as outras entidades (Exames e Compartilhamentos) e ordenando por data. Ainda assim, mencionamos essa possível entidade caso fosse desejável um log mais abrangente de atividades. Neste projeto, podemos tratar a timeline de forma lógica sem um *Event* dedicado.

- **Outras Entidades:** Poderiam existir entidades adicionais dependendo das extensões do sistema – por exemplo, uma entidade Sessão (para controlar logins ativos), ou Notificações/Lembretes (caso implementemos lembretes de exames periódicos ou vacinação, etc.), ou mesmo uma entidade Hospital/Clínica caso médicos pertençam a instituições. Contudo, nenhuma dessas é estritamente necessária no escopo atual. Vamos nos concentrar em Usuário, Exame e Compartilhamento, que cobrem os requisitos propostos.

Sprint	Período (Datas)	Foco Principal	Entregáveis Realistas (Objetivos da Sprint)	Requisitos Funcionais (RF) Cobertos
0	18 a 24 de Dezembro	Configuração e Fundação	1. Configuração do ambiente de desenvolvimento (Spring Boot, PostgreSQL). 2. Modelagem básica do banco de dados (tabelas Usuário e Exame). 3. Tela de Cadastro de Usuário (apenas perfil Paciente) e Login funcional.	RF001 (Cadastro Básico)
1	25 de Dezembro a 1 de Janeiro	Upload e Armazenamento	1. Finalização do Cadastro (perfis Médico e Laboratório). 2. Interface de Upload de Exames (RF002) com campos de metadados (título, data,	RF001 (Completo), RF002, RF004

			categoria). 3. Implementação do Armazenamento Seguro de Arquivos (RF004) no <i>backend</i> .	
2	2 a 8 de Janeiro	Visualização e Organização	1. Listagem de Exames do Paciente. 2. Visualização do Exame (RF005) para o paciente (exibição do PDF/Imagem). 3. Implementação de Filtros Básicos (RF007) na listagem (por data e categoria).	RF005 (Paciente), RF007
3	9 a 15 de Janeiro	Compartilhamento e Validação	1. Implementação do Compartilhamento Controlado (RF008): Paciente pode compartilhar um exame com um Médico/Laboratório. 2. Acesso do Médico/Laboratório aos exames compartilhados. 3. Funcionalidade de Validação Manual (RF003) do exame pelo Médico/Laboratório.	RF003, RF008
4	16 a 22 de Janeiro	Finalização e Entrega	1. Linha do Tempo de Eventos (RF009) simples (listando uploads e validações). 2. Ajustes finais de Usabilidade (RNF003) e Compatibilidade (RNF004) (<i>front-end</i>). 3. Documentação Final do Projeto (Relatório Técnico e Slides de Apresentação).	RF009, RNF003, RNF004

